

Power Query M

Formula Reference Guide

Based on *The Definitive Guide to Power Query M*
Covers: Data Types & Values • List, Record & Table Functions
Text & Number Functions • Date/Time Functions

Convention	Meaning
Blue box	Syntax / formula signature
Green box	Example with result
LABEL	Section heading within each entry

Chapter 1 — Data Types & Values

Value.Type

INTENDED USE

Returns the type of a given value. Useful for inspecting or validating data.

SYNTAX

```
Value.Type( value as any ) as type
```

PARAMETERS

value: Any M value (number, text, date, list, record, table, etc.).

EXAMPLE

```
Value.Type( 42 )  
// Result: type number
```

Type.Is

INTENDED USE

Checks whether a value's type is compatible with a given type. Returns true/false.

SYNTAX

```
Type.Is( value as any, expectedType as type ) as logical
```

PARAMETERS

value: The value to test. **expectedType:** The M type to check against (e.g., type text).

EXAMPLE

```
Type.Is( Value.Type("hello"), type text )  
// Result: true
```

Value.Is

INTENDED USE

Determines whether a value conforms to the specified type. Similar to Type.Is but operates directly on values.

SYNTAX

```
Value.Is( value as any, expectedType as type ) as logical
```

PARAMETERS

value: The value to evaluate. **expectedType:** The target type.

EXAMPLE

```
Value.Is( 3.14, type number )  
// Result: true
```

Value.ReplaceType

INTENDED USE

Returns the value with its type replaced by the specified type. Used to ascribe a type to a value without converting it.

SYNTAX

```
Value.ReplaceType( value as any, ascribedType as type ) as any
```

PARAMETERS

value: The original value. **ascribedType:** The new type to apply.

EXAMPLE

```
Value.ReplaceType( "2024-01-01", type date )  
// Result: Ascribes type date to the text (does not convert)
```

Value.NativeQuery

INTENDED USE

Submits a native query to a data source (e.g., SQL string to a SQL Server connection).

SYNTAX

```
Value.NativeQuery( target as any, query as text, optional parameters as any, optional options  
as nullable record ) as any
```

PARAMETERS

target: Data source connection. **query:** Native query string. **parameters:** Optional query parameters. **options:** Optional options record.

EXAMPLE

```
Value.NativeQuery( Sql.Database("server","db"), "SELECT * FROM Sales" )  
// Result: Returns the query result as a table
```

Chapter 2 — Text & Number Functions

Text.Combine

INTENDED USE

Concatenates a list of text values into a single string, with an optional delimiter.

SYNTAX

```
Text.Combine( texts as list, optional separator as nullable text ) as text
```

PARAMETERS

texts: A list of text values. **separator**: Optional string inserted between each value.

EXAMPLE

```
Text.Combine( {"Power", "Query", "M"}, " " )  
// Result: "Power Query M"
```

Text.Contains

INTENDED USE

Returns true if a string contains the specified substring. Case-sensitive by default.

SYNTAX

```
Text.Contains( text as nullable text, substring as text, optional comparer as nullable  
function ) as nullable logical
```

PARAMETERS

text: The string to search. **substring**: The value to find. **comparer**: Optional comparer (e.g., `Comparer.OrdinalIgnoreCase`).

EXAMPLE

```
Text.Contains( "Power Query", "Query" )  
// Result: true
```

Text.Replace

INTENDED USE

Replaces all occurrences of a substring within a text value with a new value.

SYNTAX

```
Text.Replace( text as nullable text, old as text, new as text ) as nullable text
```

PARAMETERS

text: The source string. **old**: Substring to replace. **new**: Replacement string.

EXAMPLE

```
Text.Replace( "Hello World", "World", "M" )  
// Result: "Hello M"
```

Text.Split

INTENDED USE

Splits a text string into a list using the specified delimiter.

SYNTAX

```
Text.Split( text as text, separator as text ) as list
```

PARAMETERS

text: The string to split. **separator**: The delimiter string.

EXAMPLE

```
Text.Split( "a,b,c", "," )  
// Result: {"a", "b", "c"}
```

Text.Length

INTENDED USE

Returns the number of characters in a text string.

SYNTAX

```
Text.Length( text as nullable text ) as nullable number
```

PARAMETERS

text: The string to measure.

EXAMPLE

```
Text.Length( "Power Query" )  
// Result: 11
```

Text.Start / Text.End

INTENDED USE

Extracts a specified number of characters from the beginning or end of a string.

SYNTAX

```
Text.Start( text as nullable text, count as number ) as nullable text  
Text.End( text as nullable text, count as number ) as nullable text
```

PARAMETERS

text: Source string. **count**: Number of characters to extract.

EXAMPLE

```
Text.Start( "PowerQuery", 5 )  
// Result: "Power"
```

Text.Middle

INTENDED USE

Extracts a substring starting at a given position for a specified length.

SYNTAX

```
Text.Middle( text as nullable text, start as number, optional count as nullable number ) as nullable text
```

PARAMETERS

text: Source string. **start**: 0-based start index. **count**: Optional number of chars.

EXAMPLE

```
Text.Middle( "PowerQuery", 5, 5 )  
// Result: "Query"
```

Text.Upper / Text.Lower / Text.Proper

INTENDED USE

Converts text to uppercase, lowercase, or title case respectively.

SYNTAX

```
Text.Upper( text as nullable text ) as nullable text  
Text.Lower( text as nullable text ) as nullable text  
Text.Proper( text as nullable text ) as nullable text
```

PARAMETERS

text: The string to transform.

EXAMPLE

```
Text.Proper( "power query m" )  
// Result: "Power Query M"
```

Text.Trim / Text.TrimStart / Text.TrimEnd

INTENDED USE

Removes leading and/or trailing whitespace (or specified characters) from a string.

SYNTAX

```
Text.Trim( text as nullable text, optional trim as any ) as nullable text
```

PARAMETERS

text: Source string. **trim**: Optional character or list of characters to trim (defaults to whitespace).

EXAMPLE

```
Text.Trim( " Hello " )  
// Result: "Hello"
```

Text.PadStart / Text.PadEnd

INTENDED USE

Pads a string to a specified length by adding characters at the start or end.

SYNTAX

```
Text.PadStart( text as nullable text, count as number, optional character as nullable text ) as nullable text
```

PARAMETERS

text: Source string. **count**: Target total length. **character**: Padding character (default is space).

EXAMPLE

```
Text.PadStart( "42", 6, "0" )  
// Result: "000042"
```

Number Functions

Number.Round

INTENDED USE

Rounds a number to the specified number of decimal places.

SYNTAX

```
Number.Round( number as nullable number, optional digits as nullable number, optional  
roundingMode as nullable number ) as nullable number
```

PARAMETERS

number: Value to round. **digits**: Decimal places (default 0). **roundingMode**: Optional rounding mode constant.

EXAMPLE

```
Number.Round( 3.14159, 2 )  
// Result: 3.14
```

Number.Abs

INTENDED USE

Returns the absolute (non-negative) value of a number.

SYNTAX

```
Number.Abs( number as nullable number ) as nullable number
```

PARAMETERS

number: The numeric value.

EXAMPLE

```
Number.Abs( -15 )  
// Result: 15
```

Number.Mod

INTENDED USE

Returns the remainder after dividing one number by another.

SYNTAX

```
Number.Mod( number as nullable number, divisor as nullable number ) as nullable number
```

PARAMETERS

number: The dividend. **divisor**: The divisor.

EXAMPLE

```
Number.Mod( 17, 5 )  
// Result: 2
```

Number.Power

INTENDED USE

Raises a number to the specified power.

SYNTAX

```
Number.Power( number as nullable number, power as nullable number ) as nullable number
```

PARAMETERS

number: The base. **power:** The exponent.

EXAMPLE

```
Number.Power( 2, 10 )  
// Result: 1024
```

Chapter 3 — Date / Time Functions

Date.From

INTENDED USE

Converts a value (text, number, datetime) into a date.

SYNTAX

```
Date.From( value as any, optional culture as nullable text ) as nullable date
```

PARAMETERS

value: The value to convert. **culture**: Optional locale string (e.g., "en-US").

EXAMPLE

```
Date.From( "2024-03-15" )  
// Result: #date(2024, 3, 15)
```

Date.Year / Date.Month / Date.Day

INTENDED USE

Extracts the year, month, or day component from a date value.

SYNTAX

```
Date.Year( date as nullable date ) as nullable number Date.Month( date as nullable date ) as  
nullable number Date.Day( date as nullable date ) as nullable number
```

PARAMETERS

date: A date value.

EXAMPLE

```
Date.Year( #date(2024, 6, 15) )  
// Result: 2024
```

Date.AddDays / Date.AddMonths / Date.AddYears

INTENDED USE

Adds the specified number of days, months, or years to a date.

SYNTAX

```
Date.AddDays( date as nullable date, numberOfDays as number ) as nullable date  
Date.AddMonths( date as nullable date, numberOfMonths as number ) as nullable date
```

PARAMETERS

date: The starting date. **numberOfDays/Months/Years**: Integer offset (negative moves backward).

EXAMPLE

```
Date.AddMonths( #date(2024, 1, 31), 1 )  
// Result: #date(2024, 2, 29)
```

Date.DayOfWeek

INTENDED USE

Returns the day of the week (0=Sunday by default) for a given date.

SYNTAX

```
Date.DayOfWeek( date as nullable date, optional firstDayOfWeek as nullable number ) as nullable number
```

PARAMETERS

date: A date value. **firstDayOfWeek**: Optional Day constant (e.g., Day.Monday).

EXAMPLE

```
Date.DayOfWeek( #date(2024, 3, 15), Day.Monday )  
// Result: 4 // Friday
```

Date.DayOfYear

INTENDED USE

Returns the day number within the year (1–366) for a given date.

SYNTAX

```
Date.DayOfYear( date as nullable date ) as nullable number
```

PARAMETERS

date: A date value.

EXAMPLE

```
Date.DayOfYear( #date(2024, 12, 31) )  
// Result: 366
```

Date.WeekOfYear / Date.WeekOfMonth

INTENDED USE

Returns the week number within the year or month.

SYNTAX

```
Date.WeekOfYear( date as nullable date, optional firstDayOfWeek as nullable number ) as nullable number
```

PARAMETERS

date: A date value. **firstDayOfWeek**: Optional first-day constant.

EXAMPLE

```
Date.WeekOfYear( #date(2024, 6, 15) )  
// Result: 24
```

Date.StartOfMonth / Date.EndOfMonth

INTENDED USE

Returns the first or last day of the month containing the given date.

SYNTAX

```
Date.StartOfMonth( date as nullable date ) as nullable date Date.EndOfMonth( date as nullable date ) as nullable date
```

PARAMETERS

date: A date value.

EXAMPLE

```
Date.EndOfMonth( #date(2024, 2, 1) )  
// Result: #date(2024, 2, 29)
```

Date.StartOfQuarter / Date.EndOfQuarter

INTENDED USE

Returns the first or last day of the quarter containing the given date.

SYNTAX

```
Date.StartOfQuarter( date as nullable date ) as nullable date
```

PARAMETERS

date: A date value.

EXAMPLE

```
Date.StartOfQuarter( #date(2024, 8, 20) )  
// Result: #date(2024, 7, 1)
```

DateTime.LocalNow / DateTime.FixedLocalNow

INTENDED USE

Returns the current local date/time. FixedLocalNow returns the same value throughout a single query evaluation.

SYNTAX

```
DateTime.LocalNow() as datetime DateTime.FixedLocalNow() as datetime
```

PARAMETERS

No parameters.

EXAMPLE

```
DateTime.LocalNow()  
// Result: Current datetime at time of refresh
```

Duration.From

INTENDED USE

Creates a duration value from a number of days (or converts another value).

SYNTAX

```
Duration.From( value as any ) as nullable duration
```

PARAMETERS

value: A number (interpreted as days) or compatible value.

EXAMPLE

```
Duration.From( 1.5 )  
// Result: #duration(1, 12, 0, 0) // 1 day 12 hours
```

Duration.Days / Duration.Hours / Duration.TotalHours

INTENDED USE

Extracts the days or hours component from a duration, or returns the total in a specific unit.

SYNTAX

```
Duration.Days( duration as nullable duration ) as nullable number  
Duration.TotalHours( duration as nullable duration ) as nullable number
```

PARAMETERS

duration: A duration value.

EXAMPLE

```
Duration.TotalHours( #duration(1, 6, 0, 0) )  
// Result: 30
```

Chapter 4 — List Functions

List.Count

INTENDED USE

Returns the number of items in a list.

SYNTAX

```
List.Count( list as list ) as number
```

PARAMETERS

list: Any M list value.

EXAMPLE

```
List.Count( {1, 2, 3, 4, 5} )  
// Result: 5
```

List.Sum / List.Average / List.Min / List.Max

INTENDED USE

Computes the sum, average, minimum, or maximum of a numeric list.

SYNTAX

```
List.Sum( list as list, optional precision as nullable number ) as any  
List.Average( list as list ) as any  
List.Min( list as list ) as any  
List.Max( list as list ) as any
```

PARAMETERS

list: A list of comparable values. **precision:** Optional for Sum.

EXAMPLE

```
List.Average( {10, 20, 30, 40} )  
// Result: 25
```

List.Select

INTENDED USE

Filters a list by returning only the items that satisfy a condition function.

SYNTAX

```
List.Select( list as list, selection as function ) as list
```

PARAMETERS

list: The source list. **selection:** A function that takes an item and returns logical.

EXAMPLE

```
List.Select( {1,2,3,4,5,6}, each _ > 3 )  
// Result: {4, 5, 6}
```

List.Transform

INTENDED USE

Applies a transformation function to each element of a list, returning a new list.

SYNTAX

```
List.Transform( list as list, transform as function ) as list
```

PARAMETERS

list: The source list. **transform**: Function applied to each item.

EXAMPLE

```
List.Transform( {1,2,3,4}, each _ * 2 )  
// Result: {2, 4, 6, 8}
```

List.Accumulate

INTENDED USE

Reduces a list to a single value by repeatedly applying an accumulator function. Similar to fold/reduce in functional programming.

SYNTAX

```
List.Accumulate( list as list, seed as any, accumulator as function ) as any
```

PARAMETERS

list: Source list. **seed**: Initial accumulator value. **accumulator**: Function(state, current) => next state.

EXAMPLE

```
List.Accumulate( {1,2,3,4}, 0, (state, current) => state + current )  
// Result: 10
```

List.Combine

INTENDED USE

Concatenates multiple lists into a single list.

SYNTAX

```
List.Combine( lists as list ) as list
```

PARAMETERS

lists: A list of lists to merge.

EXAMPLE

```
List.Combine( {{1,2}, {3,4}, {5}} )  
// Result: {1, 2, 3, 4, 5}
```

List.Distinct

INTENDED USE

Returns a list with duplicate values removed.

SYNTAX

```
List.Distinct( list as list, optional equationCriteria as any ) as list
```

PARAMETERS

list: The source list. **equationCriteria**: Optional custom equality.

EXAMPLE

```
List.Distinct( {1, 2, 2, 3, 3, 3} )  
// Result: {1, 2, 3}
```

List.Sort

INTENDED USE

Sorts a list in ascending or descending order.

SYNTAX

```
List.Sort( list as list, optional comparisonCriteria as any ) as list
```

PARAMETERS

list: Source list. **comparisonCriteria**: Optional — use Order.Descending or a custom comparer.

EXAMPLE

```
List.Sort( {5,3,1,4,2}, Order.Descending )  
// Result: {5, 4, 3, 2, 1}
```

List.Contains

INTENDED USE

Returns true if the list contains the specified value.

SYNTAX

```
List.Contains( list as list, value as any, optional equationCriteria as any ) as logical
```

PARAMETERS

list: The list to search. **value**: The value to find.

EXAMPLE

```
List.Contains( {"a","b","c"}, "b" )  
// Result: true
```

List.Generate

INTENDED USE

Generates a list by iteratively producing values using init, condition, and next functions.

SYNTAX

```
List.Generate( initial as function, condition as function, next as function, optional  
selector as nullable function ) as list
```

PARAMETERS

initial: ()=>seed. **condition**: (x)=>logical. **next**: (x)=>next x. **selector**: Optional transform per item.

EXAMPLE

```
List.Generate( ()=>1, each _ <= 5, each _ + 1 )  
// Result: {1, 2, 3, 4, 5}
```

List.Numbers

INTENDED USE

Generates a list of sequential numbers starting from a base value.

SYNTAX

```
List.Numbers( start as number, count as number, optional increment as nullable number ) as  
list
```

PARAMETERS

start: First value. **count**: How many numbers. **increment**: Step size (default 1).

EXAMPLE

```
List.Numbers( 0, 5, 10 )  
// Result: {0, 10, 20, 30, 40}
```

Chapter 5 — Record Functions

Record.Field

INTENDED USE

Returns the value of a named field from a record.

SYNTAX

```
Record.Field( record as record, field as text ) as any
```

PARAMETERS

record: The source record. **field**: The field name as text.

EXAMPLE

```
Record.Field( [Name="Alice", Age=30], "Age" )  
// Result: 30
```

Record.FieldNames

INTENDED USE

Returns a list of the field names in a record.

SYNTAX

```
Record.FieldNames( record as record ) as list
```

PARAMETERS

record: The source record.

EXAMPLE

```
Record.FieldNames( [A=1, B=2, C=3] )  
// Result: {"A", "B", "C"}
```

Record.FieldValues

INTENDED USE

Returns a list of values from all fields in a record, in field order.

SYNTAX

```
Record.FieldValues( record as record ) as list
```

PARAMETERS

record: The source record.

EXAMPLE

```
Record.FieldValues( [X=10, Y=20] )  
// Result: {10, 20}
```

Record.AddField

INTENDED USE

Returns a new record with an additional field added.

SYNTAX

```
Record.AddField( record as record, fieldName as text, value as any, optional delayed as nullable logical ) as record
```

PARAMETERS

record: Original record. **fieldName**: New field name. **value**: New field value.

EXAMPLE

```
Record.AddField( [A=1, B=2], "C", 3 )  
// Result: [A=1, B=2, C=3]
```

Record.RemoveFields

INTENDED USE

Returns a new record with the specified field(s) removed.

SYNTAX

```
Record.RemoveFields( record as record, fields as any ) as record
```

PARAMETERS

record: Original record. **fields**: Text field name or list of names to remove.

EXAMPLE

```
Record.RemoveFields( [A=1, B=2, C=3], "B" )  
// Result: [A=1, C=3]
```

Record.RenameFields

INTENDED USE

Returns a new record with specified fields renamed.

SYNTAX

```
Record.RenameFields( record as record, renames as list ) as record
```

PARAMETERS

record: Original record. **renames**: List of {old, new} name pairs.

EXAMPLE

```
Record.RenameFields( [Nm="Bob", Ag=25], [{"Nm","Name"}, {"Ag","Age"}] )  
// Result: [Name="Bob", Age=25]
```

Record.Combine

INTENDED USE

Merges a list of records into a single record. Later records override earlier ones on conflicts.

SYNTAX

```
Record.Combine( records as list ) as record
```

PARAMETERS

records: A list of record values.

EXAMPLE

```
Record.Combine( {[A=1,B=2], [B=99,C=3]} )  
// Result: [A=1, B=99, C=3]
```

Record.ToTable

INTENDED USE

Converts a record into a two-column table with Name and Value columns.

SYNTAX

```
Record.ToTable( record as record ) as table
```

PARAMETERS

record: The record to convert.

EXAMPLE

```
Record.ToTable( [City="Tokyo", Pop=14000000] )  
// Result: Table with rows: {Name="City",Value="Tokyo"}, {Name="Pop",Value=14000000}
```

Chapter 6 — Table Functions

Table.FromRecords

INTENDED USE

Creates a table from a list of records where each record becomes a row.

SYNTAX

```
Table.FromRecords( records as list, optional columns as any, optional missingField as nullable number ) as table
```

PARAMETERS

records: List of records. **columns:** Optional column names or type. **missingField:** How to handle missing fields.

EXAMPLE

```
Table.FromRecords( {[Name="Ana",Score=95],[Name="Ben",Score=87]} )  
// Result: Table: Name | Score
```

Table.AddColumn

INTENDED USE

Adds a new column to a table by applying a function to each row.

SYNTAX

```
Table.AddColumn( table as table, newColumnName as text, columnGenerator as function, optional columnType as nullable type ) as table
```

PARAMETERS

table: Source table. **newColumnName:** Name for new column. **columnGenerator:** Row-level function. **columnType:** Optional type.

EXAMPLE

```
Table.AddColumn( Sales, "Tax", each [Amount] * 0.1, type number )  
// Result: Adds Tax column = Amount * 0.1
```

Table.SelectRows

INTENDED USE

Filters a table by keeping only rows that satisfy a condition.

SYNTAX

```
Table.SelectRows( table as table, condition as function ) as table
```

PARAMETERS

table: Source table. **condition:** Function taking a row record, returning logical.

EXAMPLE

```
Table.SelectRows( Sales, each [Amount] > 1000 )  
// Result: Returns rows where Amount > 1000
```

Table.SelectColumns

INTENDED USE

Returns a table with only the specified columns.

SYNTAX

```
Table.SelectColumns( table as table, columns as any, optional missingField as nullable number ) as table
```

PARAMETERS

table: Source table. **columns:** Column name or list of names.

EXAMPLE

```
Table.SelectColumns( Customers, {"Name", "Email"} )  
// Result: Table with only Name and Email columns
```

Table.RemoveColumns

INTENDED USE

Returns a table with the specified columns removed.

SYNTAX

```
Table.RemoveColumns( table as table, columns as any, optional missingField as nullable number ) as table
```

PARAMETERS

table: Source table. **columns:** Column name or list of names to drop.

EXAMPLE

```
Table.RemoveColumns( Orders, {"TempCol", "RowNum"} )  
// Result: Table without TempCol and RowNum
```

Table.RenameColumns

INTENDED USE

Returns a table with specified columns renamed.

SYNTAX

```
Table.RenameColumns( table as table, renames as list, optional missingField as nullable number ) as table
```

PARAMETERS

table: Source table. **renames:** List of {old name, new name} pairs.

EXAMPLE

```
Table.RenameColumns( Data, {"Cust_ID", "CustomerID"} )  
// Result: Column Cust_ID is now CustomerID
```

Table.Sort

INTENDED USE

Sorts a table by one or more columns with optional direction.

SYNTAX

```
Table.Sort( table as table, comparisonCriteria as any ) as table
```

PARAMETERS

table: Source table. **comparisonCriteria:** Column name, or list of {column, Order} pairs.

EXAMPLE

```
Table.Sort( Sales, {{"Date", Order.Ascending}, {"Amount", Order.Descending}} )  
// Result: Sorted by Date asc, then Amount desc
```

Table.Group

INTENDED USE

Groups rows by key columns and applies aggregation functions.

SYNTAX

```
Table.Group( table as table, key as any, aggregatedColumns as list, optional groupKind as  
nullable number, optional comparer as nullable function ) as table
```

PARAMETERS

table: Source. **key:** Column(s) to group by. **aggregatedColumns:** List of {name, function} pairs.

EXAMPLE

```
Table.Group( Sales, "Region", {{"Total", each List.Sum([Amount]), type number}} )  
// Result: One row per Region with summed Amount
```

Table.Join

INTENDED USE

Joins two tables based on matching key columns. Supports inner, left outer, right outer, and full outer joins.

SYNTAX

```
Table.Join( table1 as table, key1 as any, table2 as table, key2 as any, optional joinKind as  
nullable number, optional joinAlgorithm as nullable number, optional keyEqualityComparers as  
nullable list ) as table
```

PARAMETERS

table1/table2: Tables to join. **key1/key2:** Join keys. **joinKind:** JoinKind.Inner, LeftOuter, etc.

EXAMPLE

```
Table.Join( Orders, "CustID", Customers, "ID", JoinKind.LeftOuter )  
// Result: Left outer join on CustID = ID
```

Table.NestedJoin

INTENDED USE

Joins two tables and returns the matching rows from the second table as a nested table column.

SYNTAX

```
Table.NestedJoin( table1 as table, key1 as any, table2 as any, key2 as any, newColumnName as text, optional joinKind as nullable number ) as table
```

PARAMETERS

table1/table2: Tables to join. **key1/key2:** Join keys. **newColumnName:** Name for nested table column.

EXAMPLE

```
Table.NestedJoin( Orders, "CustID", Customers, "ID", "CustData", JoinKind.LeftOuter )  
// Result: Adds CustData column containing nested table
```

Table.Pivot

INTENDED USE

Pivots a column by turning row values into column headers, aggregating data.

SYNTAX

```
Table.Pivot( table as table, pivotValues as list, attributeColumn as text, valueColumn as text, optional aggregationFunction as nullable function ) as table
```

PARAMETERS

table: Source. **pivotValues:** Unique values to become columns. **attributeColumn:** Column to pivot. **valueColumn:** Column with values.

EXAMPLE

```
Table.Pivot( Data, {"Q1","Q2","Q3"}, "Quarter", "Sales", List.Sum )  
// Result: One column per quarter with summed sales
```

Table.Unpivot / Table.UnpivotOtherColumns

INTENDED USE

Unpivots specified (or all other) columns into attribute-value pairs — the reverse of pivot.

SYNTAX

```
Table.Unpivot( table as table, pivotColumns as list, attributeColumn as text, valueColumn as text ) as table  
Table.UnpivotOtherColumns( table as table, pivotColumns as list, attributeColumn as text, valueColumn as text ) as table
```

PARAMETERS

table: Source. **pivotColumns:** Columns to unpivot (or keep). **attributeColumn/valueColumn:** Names for new columns.

EXAMPLE

```
Table.UnpivotOtherColumns( Data, {"Region"}, "Quarter", "Sales" )  
// Result: Tall table with Region, Quarter, Sales columns
```

Table.TransformColumnTypes

INTENDED USE

Changes the data type of one or more columns.

SYNTAX

```
Table.TransformColumnTypes( table as table, typeTransformations as list, optional culture as nullable text ) as table
```

PARAMETERS

table: Source. **typeTransformations:** List of {column, type} pairs.

EXAMPLE

```
Table.TransformColumnTypes( Data, {"Date", type date}, {"Amount", type number} )  
// Result: Columns cast to specified types
```

Table.FillDown / Table.FillUp

INTENDED USE

Fills null values in specified columns by propagating the previous (or next) non-null value.

SYNTAX

```
Table.FillDown( table as table, columns as list ) as table  
Table.FillUp( table as table, columns as list ) as table
```

PARAMETERS

table: Source table. **columns:** List of column names to fill.

EXAMPLE

```
Table.FillDown( Data, {"Category"} )  
// Result: Nulls in Category replaced by last non-null above
```

Table.Buffer

INTENDED USE

Forces a table to be loaded into memory and held there, preventing repeated evaluation. Useful for performance when a table is referenced multiple times.

SYNTAX

```
Table.Buffer( table as table, optional options as nullable record ) as table
```

PARAMETERS

table: Source table.

EXAMPLE

```
let Buffered = Table.Buffer( MyTable ) in Buffered  
// Result: MyTable is fully loaded into memory once
```

Chapter 7 — Control Flow & Advanced M

if ... then ... else

INTENDED USE

Conditional expression returning one of two values based on a logical test.

SYNTAX

```
if then else
```

PARAMETERS

condition: A logical expression. **trueValue/falseValue:** Any M expression.

EXAMPLE

```
if [Score] >= 50 then "Pass" else "Fail"  
// Result: "Pass" or "Fail" depending on Score
```

let ... in

INTENDED USE

Defines named variables (bindings) for use within a single expression. The fundamental M expression structure.

SYNTAX

```
let Name1 = expression1, Name2 = expression2 in Name2
```

PARAMETERS

Each binding is a name = expression pair. The **in** clause is the returned value.

EXAMPLE

```
let x = 5, y = x * 2 in y  
// Result: 10
```

each _ (shorthand function syntax)

INTENDED USE

Creates an anonymous single-parameter function. The underscore **_** refers to the current item.

SYNTAX

```
each // Equivalent to: (_) =>
```

PARAMETERS

_: The implicit parameter representing the current item in context.

EXAMPLE

```
List.Select( {1,2,3,4,5}, each _ > 3 )  
// Result: {4, 5}
```

try ... otherwise

INTENDED USE

Attempts an expression and returns a fallback value if an error occurs.

SYNTAX

```
try otherwise
```

PARAMETERS

expression: The expression that might fail. **fallbackValue**: Returned on error.

EXAMPLE

```
try Number.FromText("abc") otherwise 0
// Result: 0
```

error

INTENDED USE

Raises a user-defined error with a message and optional detail record.

SYNTAX

```
error "message" error Error.Record( "ErrorType", "message", detail )
```

PARAMETERS

message: Error description text. **detail**: Optional additional info.

EXAMPLE

```
if [Age] < 0 then error "Age cannot be negative" else [Age]
// Result: Raises error if Age is negative
```

Function definition (x) => expression

INTENDED USE

Defines a named or anonymous function with one or more parameters.

SYNTAX

```
( parameter1 as type, parameter2 as type ) as returnType => expression
```

PARAMETERS

Parameters optionally include type annotations. The return type annotation is also optional.

EXAMPLE

```
let Double = (n as number) as number => n * 2 in Double(7)
// Result: 14
```

@ (recursive function reference)

INTENDED USE

Allows a function to call itself recursively using the @ symbol within a let binding.

SYNTAX

```
let FuncName = (x) => if then else FuncName( @FuncName, nextX ) in FuncName
```

PARAMETERS

@: Refers to the current function binding, enabling recursion.

EXAMPLE

```
let Fact = (n) => if n <= 1 then 1 else n * @Fact(n-1) in Fact(5)
// Result: 120
```

Table.AddIndexColumn

INTENDED USE

Adds an index column to a table with customizable start and increment values.

SYNTAX

```
Table.AddIndexColumn( table as table, newColumnName as text, optional initialValue as
nullable number, optional increment as nullable number, optional columnType as nullable type
) as table
```

PARAMETERS

table: Source. **newColumnName**: Index column name. **initialValue**: Default 0. **increment**: Default 1.

EXAMPLE

```
Table.AddIndexColumn( Orders, "RowNum", 1, 1, Int64.Type )
// Result: Adds RowNum column: 1, 2, 3, ...
```

Table.FromList

INTENDED USE

Creates a single-column (or multi-column) table from a list by applying a splitter function.

SYNTAX

```
Table.FromList( list as list, optional splitter as nullable function, optional columns as
any, optional default as nullable any, optional extraValues as nullable number ) as table
```

PARAMETERS

list: Source list. **splitter**: Function to split each item into fields.

EXAMPLE

```
Table.FromList( {"a,1","b,2"}, Splitter.SplitByNothing(), {"Row"} )
// Result: Single-column table from list
```

This reference guide covers the core Power Query M functions. For the complete function library, refer to Microsoft's official M Language Specification and docs.microsoft.com/power-query.